
PRACTICAL - 4

Implementation and Time Analysis of Factorial using Iterative and Recursive Methods

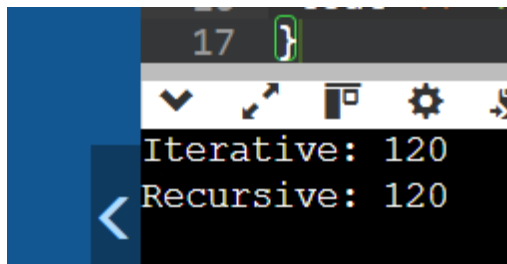
Aim of the Practical:

To implement factorial iteratively and recursively and compare their time and space complexities.

Program:

```
#include <iostream>
using namespace std;
long long factorialIterative(int n) {
    long long fact = 1;
    for (int i = 2; i <= n; i++)
        fact *= i;
    return fact;
}
long long factorialRecursive(int n) {
    if (n <= 1) return 1;
    return n * factorialRecursive(n - 1);
}
int main() {
    int n = 5;
    cout << "Iterative: " << factorialIterative(n) << endl;
    cout << "Recursive: " << factorialRecursive(n) << endl;
}
```

Output:



Time Analysis:

Iterative Method - Derivation:

- The loop runs exactly $(n - 1)$ times.
- Because each iteration performs constant work, the time complexity is $\Theta(n)$.

Recursive Method:

- The function calls itself with a recurrence of $T_{rec}(n) = T_{rec}(n - 1) + c$.
- Unrolling this recurrence step-by-step yields a time complexity of $\Theta(n)$

Final Time Analysis:

- Iterative Time: $\Theta(n)$, Space: $\Theta(1)$.
- Recursive Time: $\Theta(n)$, Space: $\Theta(n)$.

Result:

Both methods compute factorial correctly. Their time is linear, but recursion uses linear stack space.